# Python Placement Question Bank

Basic Python - Coding Programs and Interview Theory

**181** coding programs with question + answer
**63** interview theory questions with answers
**19** topics, from print() to OOP - no DSA

Prepared for **Suryakanta Pradhan**
MCA - Placement Preparation 2026-27
August 2026

# How to use this book

## Part A - Coding programs

Every program is written as a question first, then the answer. Cover the code with your hand, try to write it yourself, and only then look. The yellow Note under a program is the mistake most beginners make in that question - read those twice.

## Part B - Theory questions

These are asked in the technical round of TCS, Infosys, Wipro, Cognizant and Accenture. Practise saying each answer out loud in 2-3 sentences; do not memorise word by word.

## How to revise

Day 1: Part A topics 1-5. Day 2: topics 6-10. Day 3: topics 11-15. Day 4: all of Part B. Day 5: only the yellow Notes and the rapid-fire section. Then repeat the cycle - the second round takes half the time.

## Run the code

Open the companion file python_placement_answers.py in VS Code. Every answer is a numbered function, so you can run and change any program while you read.

# Contents

**Part A - Coding Programs** 4

1. Basics, Input / Output and Operators . . . 5
2. Decision Making (if / elif / else) . . . 8
3. Loops and Patterns . . . 12
4. Number Programs - Digits, Factorial, Reverse . . . 16
5. Number Programs - Primes, Factors, GCD, LCM . . . 19
6. Strings - Basics and Common Programs . . . 23
7. Strings - Slicing, Escape Characters and Word Programs . . . 26
8. Lists - Methods and Basic Programs . . . 29
9. Lists - Slicing, Copying and Comprehensions . . . 33
10. Tuples, Sets and Dictionaries . . . 36
11. Functions - Arguments and Return Values . . . 39
12. Lambda, Decorators, Generators and Recursion . . . 43
13. Exception Handling and File Handling . . . 47
14. Object Oriented Programming (Basics) . . . 50
15. Modules and Practical Extras . . . 54

**Part B - Interview Theory** 56

A. Language Basics and Data Types . . . 57
B. Loops, Functions and Advanced Concepts . . . 59
C. OOP, Errors, Files and Memory . . . 61
D. Rapid-fire Differences and Output Questions . . . 64

# PART A - CODING PROGRAMS (Question and Answer)

Basic Python only - printing, loops, numbers, strings, lists, tuples, sets, dictionaries, functions, file handling and simple OOP. Data structures and algorithm questions are deliberately left out.

# 1. Basics, Input / Output and Operators

### Q1. Write a program to print a message on the screen.

```
print("Hello, World!")
```

> ***Note:*** *print() adds a newline at the end by default. Use end="" to stop that.*

### Q2. Take two numbers from the user and print their sum.

```
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))
total = num1 + num2
print(f"Sum of the two numbers is {total}")
```

> ***Note:*** *input() always returns a string, so int() conversion is compulsory before adding.*

### Q3. Swap two numbers using a third (temporary) variable.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
temp = a
a = b
b = temp
print("After swapping: a =", a, ", b =", b)
```

### Q4. Swap two numbers without using a third variable.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
a, b = b, a          # tuple unpacking - the Pythonic way
print("After swapping: a =", a, ", b =", b)
```

> ***Note:*** *Interviewers also accept a = a + b; b = a - b; a = a - b. The tuple version is preferred.*

### Q5. Convert a temperature from Celsius to Fahrenheit.

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9 / 5) + 32
print(f"{celsius} C = {fahrenheit} F")
```

### Q6. Convert a temperature from Fahrenheit to Celsius.

```
fahrenheit = float(input("Enter temperature in Fahrenheit: "))
celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit} F = {celsius:.2f} C")
```

> ***Note:*** *{value:.2f} inside an f-string rounds the output to 2 decimal places.*

### Q7. Calculate simple interest and the final amount.

```
principal = float(input("Enter principal amount: "))
rate = float(input("Enter rate of interest: "))
time = float(input("Enter time in years: "))
interest = (principal * rate * time) / 100
amount = principal + interest
print(f"Simple interest = {interest}")
print(f"Final amount = {amount}")
```

### Q8. Find the area and perimeter of a circle.

```
import math

radius = float(input("Enter radius: "))
area = math.pi * radius ** 2
perimeter = 2 * math.pi * radius
print(f"Area = {area:.2f}")
print(f"Perimeter = {perimeter:.2f}")
```

***Note:*** *** is the power operator. math.pi is more accurate than writing 3.14.*

### Q9. Find the average of two numbers.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
average = (a + b) / 2
print("Average =", average)
```

***Note:*** *Brackets matter. a + b / 2 divides only b by 2 - a very common mistake.*

### Q10. Show all the arithmetic operators on two numbers.

```
a, b = 17, 5
print("Addition       :", a + b)     # 22
print("Subtraction    :", a - b)     # 12
print("Multiplication :", a * b)     # 85
print("Division       :", a / b)     # 3.4  -> always float
print("Floor division :", a // b)    # 3    -> drops the decimal
print("Modulus        :", a % b)     # 2    -> remainder
print("Power          :", a ** b)    # 1419857
```

***Note:*** */ gives a float, // gives the quotient, % gives the remainder.*

### Q11. Check the data type of a value and test it with isinstance().

```
x = 200
y = "surya"
z = [1, 2, 3]

print(type(x), type(y), type(z))
print(isinstance(x, int))       # True
print(isinstance(y, (int, str)))  # True - tuple means "any of these"
```

***Note:*** *type() tells the exact type; isinstance() also returns True for subclasses.*

### Q12. Take a full name and print it in different formats using f-strings.

```
first = input("First name: ")
last = input("Last name: ")

print(f"Full name : {first} {last}")
print(f"Initials  : {first[0].upper()}.{last[0].upper()}.")
print(f"Length    : {len(first) + len(last)} letters")
print("Full name : {} {}".format(first, last))   # older style
```

> ***Note:*** *f-strings (Python 3.6+) are the recommended way to format text.*

## 2. Decision Making (if / elif / else)

### Q13. Check whether a number is even or odd.

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Number is even")
else:
    print("Number is odd")
```

***Note:*** *The % (modulus) operator gives the remainder. Remainder 0 means even.*

### Q14. Find the largest of three numbers.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
c = int(input("Enter c: "))

if a >= b and a >= c:
    print("a is the greatest:", a)
elif b >= a and b >= c:
    print("b is the greatest:", b)
else:
    print("c is the greatest:", c)

# one-line alternative
print("Greatest =", max(a, b, c))
```

***Note:*** *Use >= instead of > so equal numbers do not fall to the wrong branch.*

### Q15. Check whether a number is positive, negative or zero.

```
num = float(input("Enter a number: "))
if num > 0:
    print("Positive number")
elif num < 0:
    print("Negative number")
else:
    print("Number is zero")
```

### Q16. Check whether a number is a multiple of 7.

```
num = int(input("Enter a number: "))
if num % 7 == 0:
    print(num, "is a multiple of 7")
else:
    print(num, "is not a multiple of 7")
```

### Q17. Check whether a year is a leap year.

```
year = int(input("Enter a year: "))
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
    print(year, "is a leap year")
else:
    print(year, "is not a leap year")
```

***Note:*** *Divisible by 4, but century years (1900, 2100) must also be divisible by 400.*

### Q18. Check voting eligibility using a nested if.

```
age = int(input("Enter your age: "))

if age >= 18:
    if age <= 100:
        print("Eligible to vote")
    else:
        print("Please check the age entered")
else:
    print("Not eligible to vote")
```

***Note:*** *An if written inside another if is called a nested if.*

### Q19. Print the grade of a student from marks (A/B/C/D/Fail).

```
marks = int(input("Enter marks out of 100: "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "Fail"

print("Grade =", grade)
```

***Note:*** *Order the conditions from highest to lowest, otherwise every mark hits the first branch.*

### Q20. Calculate a final bill price with a member discount (10%) and a coupon discount (5%).

```
def calculate_price(original_price, is_member, has_coupon):
    bill = original_price
    if is_member:
        bill *= 0.90        # 10% off for members
    if has_coupon:
        bill *= 0.95        # 5% off for coupon
    return bill

price = float(input("Enter the bill amount: "))
is_member = input("Are you a member? (yes/no): ").lower() == "yes"
has_coupon = input("Do you have a coupon? (yes/no): ").lower() == "yes"

print(f"Final price: Rs.{calculate_price(price, is_member, has_coupon):.2f}")
```

***Note:*** *Careful: bool(input()) is always True for any typed text, even "no". Compare with == instead.*

### Q21. Find the largest of two numbers using a ternary (one-line if).

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

largest = a if a > b else b
print("Largest =", largest)

print("Even" if a % 2 == 0 else "Odd")
```

> ***Note:*** *Syntax: value_if_true if condition else value_if_false.*

### Q22. Check whether a character entered by the user is a vowel or a consonant.

```
ch = input("Enter a single letter: ").lower()

if not ch.isalpha() or len(ch) != 1:
    print("Not a valid letter")
elif ch in "aeiou":
    print("Vowel")
else:
    print("Consonant")
```

> ***Note:*** *The in operator works on strings, so ch in "aeiou" replaces 5 comparisons.*

### Q23. Calculate BMI and print the category.

```
weight = float(input("Weight in kg: "))
height = float(input("Height in metres: "))

bmi = weight / (height ** 2)
print(f"BMI = {bmi:.2f}")

if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

### Q24. Write a simple calculator that takes two numbers and an operator.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
op = input("Enter operator (+ - * /): ")

if op == "+":
    print(a + b)
elif op == "-":
    print(a - b)
elif op == "*":
    print(a * b)
elif op == "/":
    print("Cannot divide by zero" if b == 0 else a / b)
else:
    print("Invalid operator")
```

> ***Note:*** *Always handle division by zero - interviewers look for this check.*

# 3. Loops and Patterns

### Q25. Print numbers from 1 to 10 using a for loop.

```
for i in range(1, 11):
    print(i, end=" ")
```

***Note:*** *range(1, 11) stops before 11, so it gives 1 to 10.*

### Q26. Print numbers from 1 to 10 using a while loop.

```
i = 1
while i <= 10:
    print(i, end=" ")
    i += 1
```

***Note:*** *Forgetting i += 1 creates an infinite loop.*

### Q27. Print the multiplication table of a number using a for loop.

```
n = int(input("Enter a number: "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

***Note:*** *Keep the print inside the loop; if it is outside you only see the last line.*

### Q28. Print the multiplication table of a number using a while loop.

```
n = int(input("Enter a number: "))
i = 1
while i <= 10:
    print(f"{n} x {i} = {n * i}")
    i += 1
```

***Note:*** *Loop on the counter (i <= 10), not on n - a common slip.*

### Q29. Find the sum of the first N natural numbers (loop and formula).

```
n = int(input("Enter N: "))

total = 0
for i in range(1, n + 1):
    total += i
print("Sum using loop    =", total)

print("Sum using formula =", n * (n + 1) // 2)
```

***Note:*** *The formula n(n+1)/2 answers it in O(1) time - mention this in an interview.*

### Q30. Print the sum of all even numbers and all odd numbers from 1 to N.

```
n = int(input("Enter N: "))
even_sum = odd_sum = 0

for i in range(1, n + 1):
    if i % 2 == 0:
        even_sum += i
    else:
        odd_sum += i

print("Even sum =", even_sum)
print("Odd sum  =", odd_sum)
```

### Q31. Print the Fibonacci series up to N terms.

```
n = int(input("Enter number of terms: "))
a, b = 0, 1

for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

***Note:*** *The series must start at 0, 1. Use _ when the loop variable is not needed.*

### Q32. Print a right-angled triangle of stars (increasing).

```
n = 5
for i in range(1, n + 1):
    print("* " * i)

# Output:
# *
# * *
# * * *
# * * * *
# * * * * *
```

***Note:*** *Multiplying a string ("* " * 3) repeats it - shorter than an inner loop.*

### Q33. Print a reverse (decreasing) triangle of stars.

```
n = 5
for i in range(n, 0, -1):
    print("* " * i)
```

***Note:*** *range(n, 0, -1) counts downwards. The third value is the step.*

### Q34. Print a number pattern using nested loops.

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()      # move to the next line

# Output:
# 1
# 1 2
# 1 2 3
# 1 2 3 4
# 1 2 3 4 5
```

> ***Note:*** *The empty print() after the inner loop is what breaks the line.*

### Q35. Print a pyramid pattern of stars.

```
n = 5
for i in range(1, n + 1):
    print(" " * (n - i) + "* " * i)

# Output:
#     *
#    * *
#   * * *
#  * * * *
# * * * * *
```

### Q36. Write the FizzBuzz program for 1 to N.

```
n = int(input("Enter N: "))

for i in range(1, n + 1):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

> ***Note:*** *Check 15 first. If 3 and 5 are checked first, FizzBuzz never prints. Very common test question.*

### Q37. Show the difference between break, continue and pass.

```
for i in range(1, 6):
    if i == 3:
        break            # stop the loop completely
    print(i, end=" ")    # 1 2
print()

for i in range(1, 6):
    if i == 3:
        continue         # skip only this turn
    print(i, end=" ")    # 1 2 4 5
print()

for i in range(1, 6):
    if i == 3:
        pass             # does nothing, just a placeholder
    print(i, end=" ")    # 1 2 3 4 5
```

> ***Note:*** *break exits, continue skips one turn, pass is only a filler statement.*

### Q38. Print all combinations of two lists using nested loops.

```
adjectives = ["red", "big", "tasty"]
fruits = ["apple", "orange", "mango"]

for adj in adjectives:
    for fruit in fruits:
        print(adj, fruit)
```

> ***Note:*** *The inner loop runs fully for every single turn of the outer loop (3 x 3 = 9 lines).*

### Q39. Write a menu-driven program that keeps running until the user exits.

```
while True:
    print("\n1. Say hello  2. Show square  3. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        print("Hello!")
    elif choice == "2":
        num = int(input("Enter a number: "))
        print("Square =", num ** 2)
    elif choice == "3":
        print("Exiting...")
        break
    else:
        print("Invalid choice, try again")
```

> ***Note:*** *while True with a break is the standard pattern for menu-driven programs.*

# 4. Number Programs - Digits, Factorial, Reverse

### Q40. Print all the digits of a number one by one (from the last digit).

```
def print_digits(n):
    while n > 0:
        digit = n % 10        # take the last digit
        print(digit, end=" ")
        n //= 10              # remove the last digit

print_digits(5875)            # 5 7 8 5
```

***Note:*** *n % 10 gives the last digit, n //= 10 removes it. This pair is used in almost every digit program.*

### Q41. Count the number of digits in a number.

```
def count_digits(n):
    count = 0
    while n > 0:
        count += 1
        n //= 10
    return count

print(count_digits(5336))     # 4
print(len(str(5336)))         # 4  (shortcut)
```

***Note:*** *Return the count after the loop finishes; printing inside the loop shows partial counts.*

### Q42. Find the sum of the digits of a number.

```
n = int(input("Enter a number: "))
total = 0

while n > 0:
    total += n % 10
    n //= 10            # NOT n // 10 - it must be reassigned

print("Sum of digits =", total)
```

***Note:*** *n // 10 alone does nothing. You must write n = n // 10 or n //= 10.*

### Q43. Reverse a number.

```
num = int(input("Enter a number: "))
original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num //= 10

print(f"Original = {original}, Reversed = {reverse}")
```

***Note:*** *reverse * 10 + digit shifts the existing digits left and adds the new one.*

### Q44. Check whether a number is a palindrome (using maths).

```
num = int(input("Enter a number: "))
original, reverse = num, 0

while num > 0:
    reverse = reverse * 10 + num % 10
    num //= 10

if original == reverse:
    print(original, "is a palindrome")
else:
    print(original, "is not a palindrome")
```

### Q45. Check whether a number is a palindrome using string slicing.

```
n = input("Enter a number: ")

if n == n[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

> ***Note:*** *[::-1] means "whole string, step -1", i.e. reversed. Works for strings too.*

### Q46. Find the factorial of a number using a for loop.

```
n = int(input("Enter a number: "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print(f"Factorial of {n} = {fact}")
```

> ***Note:*** *Start fact at 1, not 0. Print after the loop, not inside it.*

### Q47. Find the factorial of a number using a while loop, and handle negative input.

```
n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    fact, i = 1, 1
    while i <= n:
        fact *= i
        i += 1
    print(f"Factorial of {n} = {fact}")   # 0! = 1
```

> ***Note:*** *Factorial of 0 is 1 - the loop handles it automatically.*

### Q48. Find the factorial of a number using the math module.

```
import math

n = int(input("Enter a number: "))
print(math.factorial(n))
```

> ***Note:*** *Fine in real work, but interviewers usually want the loop version.*

### Q49. Check whether a number is an Armstrong number.

```
num = int(input("Enter a number: "))
temp = num
digits = len(str(num))
total = 0

while temp > 0:
    digit = temp % 10
    total += digit ** digits
    temp //= 10

if total == num:
    print(num, "is an Armstrong number")
else:
    print(num, "is not an Armstrong number")
```

> ***Note:*** *Copy num into temp first. If you destroy num in the loop, the final comparison always fails.*

### Q50. Find the digital root of a number (keep adding digits until one digit is left).

```
n = int(input("Enter a number: "))

while n > 9:
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    n = total

print("Digital root =", n)     # 9875 -> 29 -> 11 -> 2
```

# 5. Number Programs - Primes, Factors, GCD, LCM

### Q51. Check whether a number is prime (basic method).

```
num = int(input("Enter a number: "))

if num <= 1:
    print(num, "is not a prime number")
else:
    for i in range(2, num):
        if num % i == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a PRIME number")
```

***Note:*** *The else belongs to the for loop: it runs only if break never happened. This is for-else.*

### Q52. Check whether a number is prime in an optimised way (up to the square root).

```
num = int(input("Enter a number: "))

if num <= 1:
    print("Not prime")
else:
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    print("Prime" if is_prime else "Not prime")
```

***Note:*** *Checking only up to sqrt(n) reduces the work from O(n) to O(sqrt n). Say this in interviews.*

### Q53. Check whether a number is prime by counting its factors.

```
def is_prime(n):
    count = 0
    for i in range(1, n + 1):
        if n % i == 0:
            count += 1
    return count == 2      # exactly 1 and itself

print(is_prime(3))         # True
print(is_prime(4))         # False
```

***Note:*** *A prime number has exactly two factors. 1 has only one factor, so 1 is not prime.*

### Q54. Print all prime numbers from 1 to N.

```
n = int(input("Enter N: "))

for num in range(2, n + 1):
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
```

***Note:*** *Reset is_prime = True inside the outer loop for every new number.*

### Q55. Print all the factors of a number.

```
def factors(n):
    for i in range(1, n + 1):
        if n % i == 0:
            print(i, end=" ")

factors(20)      # 1 2 4 5 10 20
```

### Q56. Print all factors of a number efficiently (up to the square root).

```
import math

def factors(n):
    result = []
    for i in range(1, int(math.sqrt(n)) + 1):
        if n % i == 0:
            result.append(i)
            if i != n // i:          # avoid repeating for perfect squares
                result.append(n // i)
    return sorted(result)

print(factors(36))      # [1, 2, 3, 4, 6, 9, 12, 18, 36]
```

***Note:*** *Factors come in pairs (i, n//i), so one loop to sqrt(n) finds both.*

### Q57. Find the GCD (HCF) of two numbers using the Euclidean algorithm.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

x, y = a, b
while y != 0:
    x, y = y, x % y

print("GCD =", x)

# import math; print(math.gcd(a, b))   # built-in shortcut
```

***Note:*** *Euclid: keep replacing (x, y) with (y, x % y) until y becomes 0.*

### Q58. Find the LCM of two numbers.

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

x, y = a, b
while y != 0:
    x, y = y, x % y
gcd = x

lcm = (a * b) // gcd      # divide by the GCD, not by b
print("GCD =", gcd, " LCM =", lcm)
```

> ***Note:*** *Formula: LCM = (a * b) / GCD. Dividing by b instead of gcd is a very common error.*

### Q59. Check whether a number is a perfect number.

```
num = int(input("Enter a number: "))
total = 0

for i in range(1, num):
    if num % i == 0:
        total += i

print("Perfect number" if total == num else "Not a perfect number")
# 6 = 1 + 2 + 3, 28 = 1 + 2 + 4 + 7 + 14
```

> ***Note:*** *A perfect number equals the sum of its proper divisors (excluding itself).*

### Q60. Check whether a number is a strong number (sum of factorial of digits equals the number).

```
import math

num = int(input("Enter a number: "))
temp, total = num, 0

while temp > 0:
    total += math.factorial(temp % 10)
    temp //= 10

print("Strong number" if total == num else "Not a strong number")
# 145 = 1! + 4! + 5! = 1 + 24 + 120
```

### Q61. Convert a decimal number to binary and back.

```
n = int(input("Enter a decimal number: "))

binary = ""
temp = n
while temp > 0:
    binary = str(temp % 2) + binary   # add the new bit in front
    temp //= 2
print("Binary =", binary or "0")
print("Built-in :", bin(n)[2:])

print("Back to decimal :", int(binary, 2))
```

> ***Note:*** *bin() returns a string like '0b101', so [2:] removes the prefix. int(s, 2) converts back.*

### Q62. Find the largest and smallest digit in a number.

```
n = int(input("Enter a number: "))
largest, smallest = 0, 9

while n > 0:
    digit = n % 10
    largest = max(largest, digit)
    smallest = min(smallest, digit)
    n //= 10

print("Largest digit =", largest)
print("Smallest digit =", smallest)
```

# 6. Strings - Basics and Common Programs

### Q63. Show the most used string methods on one string.

```
a = "  I am Suryakanta  "

print(a.strip())           # removes spaces from both ends
print(a.lower())           # small letters
print(a.upper())           # capital letters
print(a.title())           # I Am Suryakanta
print(a.capitalize())      # only first letter capital
print(a.swapcase())        # upper <-> lower
print(a.replace("am", "was"))
print(a.split())           # list of words
print(a.strip().count("a"))
print(a.find("am"))        # index, or -1 if not found
print(len(a), "am" in a)
```

> ***Note:*** *String methods never change the original string; they return a new one.*

### Q64. Reverse a string (two ways).

```
s = input("Enter a string: ")

print("Slicing :", s[::-1])

rev = ""
for ch in s:
    rev = ch + rev          # put every new letter in front
print("Loop    :", rev)
```

> ***Note:*** *In the loop version write rev = ch + rev, not ch + s.*

### Q65. Check whether a string is a palindrome.

```
s = input("Enter a string: ").lower().replace(" ", "")

if s == s[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
# "Never odd or even" -> palindrome
```

> ***Note:*** *Clean the input first (lowercase, remove spaces), otherwise "Madam" fails.*

### Q66. Count the number of vowels in a string.

```
s = input("Enter a string: ")
vowels = "aeiouAEIOU"

count = 0
for ch in s:
    if ch in vowels:
        count += 1
print("Vowels =", count)

print("One-liner:", sum(1 for ch in s if ch in vowels))
```

### Q67. Count vowels, consonants, digits and spaces in a string.

```
s = input("Enter a string: ")
v = c = d = sp = 0

for ch in s:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            v += 1
        else:
            c += 1
    elif ch.isdigit():
        d += 1
    elif ch.isspace():
        sp += 1

print(f"Vowels={v} Consonants={c} Digits={d} Spaces={sp}")
```

***Note:*** *isalpha(), isdigit(), isspace() and isalnum() are frequently asked.*

### Q68. Count the frequency of each character in a string.

```
s = input("Enter a string: ")
freq = {}

for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)

# from collections import Counter
# print(Counter(s))
```

***Note:*** *dict.get(key, 0) returns 0 when the key is missing, so no KeyError.*

### Q69. Check whether two strings are anagrams.

```
s1 = input("Enter first string: ").lower()
s2 = input("Enter second string: ").lower()

if sorted(s1) == sorted(s2):
    print("Anagram")
else:
    print("Not an anagram")
# "listen" and "silent" -> anagram
```

***Note:*** *sorted() on a string returns a list of characters, so the comparison works.*

### Q70. Count the number of words in a sentence.

```
sentence = input("Enter a sentence: ")
words = sentence.split()

print("Number of words =", len(words))
print("Words :", words)
```

***Note:*** *split() with no argument splits on any amount of whitespace - safer than split(" ").*

### Q71. Find the longest word in a sentence.

```
sentence = input("Enter a sentence: ")
words = sentence.split()

longest = ""
for word in words:
    if len(word) > len(longest):
        longest = word
print("Longest word =", longest)

print("One-liner:", max(words, key=len))
```

***Note:*** *max(words, key=len) compares words by their length.*

### Q72. Reverse the order of words in a sentence.

```
sentence = input("Enter a sentence: ")

reversed_sentence = " ".join(sentence.split()[::-1])
print(reversed_sentence)
# "I am Surya" -> "Surya am I"
```

***Note:*** *split() to a list, reverse the list, then join back with a space.*

### Q73. Capitalise the first letter of every word without using title().

```
sentence = input("Enter a sentence: ")

result = " ".join(word[0].upper() + word[1:] for word in sentence.split())
print(result)
```

***Note:*** *word[0].upper() + word[1:] keeps the rest of the word as it is.*

### Q74. Remove duplicate characters from a string but keep the original order.

```
s = input("Enter a string: ")
seen = set()
result = ""

for ch in s:
    if ch not in seen:
        seen.add(ch)
        result += ch

print(result)      # "programming" -> "progamin"
```

***Note:*** *set(s) also removes duplicates but destroys the order.*

# 7. Strings - Slicing, Escape Characters and Word Programs

### Q75. Show string slicing and negative indexing.

```
s = "Suryakanta"

print(s[0])        # S       -> first character
print(s[-1])       # a       -> last character
print(s[0:5])      # Surya   -> index 0 to 4
print(s[:5])       # Surya
print(s[5:])       # kanta
print(s[:-1])      # Suryakant
print(s[::2])      # Sraat   -> every 2nd character
print(s[::-1])     # atnakayruS
```

> ***Note:*** *Format is s[start:stop:step]. stop is never included.*

### Q76. Show that strings are immutable.

```
s = "surya"

# s[0] = "S"        # TypeError - strings cannot be changed

s = "S" + s[1:]     # create a new string instead
print(s)            # Surya
```

> ***Note:*** *Immutable = cannot be changed in place. Lists are mutable, strings and tuples are not.*

### Q77. Show the common escape sequences.

```
print("Line one\nLine two")      # \n = new line
print("Name\tRoll")              # \t = tab
print("She said \"hello\"")      # \" = double quote inside quotes
print('It\'s fine')              # \' = single quote
print("Path: C:\\Users\\Admin")  # \\ = one real backslash
print(r"C:\Users\Admin")         # r"..." = raw string, no escaping
```

> ***Note:*** *Note it is \n (backslash n), not /n - that is the mistake most beginners make.*

### Q78. Check whether one string is present inside another (substring check).

```
text = "I am learning Python"
word = input("Enter a word to search: ")

if word in text:
    print("Found at index", text.find(word))
else:
    print("Not found")

# find() returns -1 if missing, index() raises an error
```

> ***Note:*** *in is the simplest membership test. find() gives the position.*

### Q79. Count the frequency of each word in a sentence.

```
sentence = input("Enter a sentence: ").lower()
freq = {}

for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1

for word, count in freq.items():
    print(word, ":", count)
```

### Q80. Remove all punctuation and spaces from a string.

```
s = "Hello, World! Python is great."

clean = "".join(ch for ch in s if ch.isalnum())
print(clean)          # HelloWorldPythonisgreat
```

***Note:*** *isalnum() keeps only letters and digits.*

### Q81. Convert a string to a list of characters and back.

```
s = "surya"

chars = list(s)
print(chars)                # a list of single characters

chars[0] = "S"              # lists ARE changeable
print("".join(chars))       # Surya
```

***Note:*** *This is the standard trick when you need to "edit" a string.*

### Q82. Check whether a string contains only digits and convert it safely.

```
s = input("Enter a number: ")

if s.isdigit():
    print("Number =", int(s) * 2)
else:
    print("Please enter digits only")
```

***Note:*** *isdigit() prevents a ValueError crash from int("abc").*

### Q83. Print the ASCII value of a character and the character of an ASCII value.

```
ch = input("Enter a character: ")

print("ASCII value of", ch, "is", ord(ch))
print("Character of 65 is", chr(65))      # A

for i in range(65, 70):
    print(chr(i), end=" ")                # A B C D E
```

***Note:*** *ord() = character to number, chr() = number to character.*

### Q84. Print each character of a string with its index.

```
s = "Python"

for index, ch in enumerate(s):
    print(index, ch)

# without enumerate
for i in range(len(s)):
    print(i, s[i])
```

***Note:*** *enumerate() gives the index and the value together - cleaner than range(len(s)).*

### Q85. Toggle the case of every character without using swapcase().

```
s = input("Enter a string: ")
result = ""

for ch in s:
    if ch.isupper():
        result += ch.lower()
    elif ch.islower():
        result += ch.upper()
    else:
        result += ch

print(result)
```

### Q86. Check whether two strings are equal, ignoring case and extra spaces.

```
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

if s1.strip().lower() == s2.strip().lower():
    print("Both strings are same")
else:
    print("Strings are different")
```

***Note:*** *Always normalise (strip + lower) before comparing user input.*

# 8. Lists - Methods and Basic Programs

### Q87. Show the most used list methods.

```
nums = [1, 5, 2, 4, 6, 3]

nums.append(7)          # add at the end
nums.insert(1, 10)      # add 10 at index 1
nums.extend([8, 9])     # add many items
print(nums)

nums.remove(2)          # remove the VALUE 2
nums.pop(0)             # remove by INDEX and return it
nums.sort()             # sort in place (ascending)
nums.sort(reverse=True) # descending
nums.reverse()          # only reverse the order
print(nums, len(nums), nums.count(4), nums.index(4))
```

***Note:*** *remove() takes a value, pop() takes an index. append() adds one item, extend() adds many.*

### Q88. Show the difference between append() and extend().

```
a = [1, 2, 3]
b = [1, 2, 3]

a.append([4, 5])
print(a)        # [1, 2, 3, [4, 5]]  -> nested list

b.extend([4, 5])
print(b)        # [1, 2, 3, 4, 5]    -> flat list
```

***Note:*** *Very common interview question. append() adds the whole object as one item.*

### Q89. Traverse a list in three different ways.

```
fruits = ["apple", "orange", "mango"]

for fruit in fruits:            # best way
    print(fruit)

for i in range(len(fruits)):    # when you need the index
    print(i, fruits[i])

i = 0
while i < len(fruits):          # while version
    print(fruits[i])
    i += 1
```

### Q90. Remove duplicate elements from a list.

```
my_list = [2, 4, 4, 2, 1, 1, 4, 3, 5, 5, 3]

print(list(set(my_list)))            # fast, but order is lost

unique = []
for item in my_list:                 # keeps the original order
    if item not in unique:
        unique.append(item)
print(unique)
```

***Note:*** *A set cannot contain duplicates, which is why list(set(x)) works.*

### Q91. Find the largest and smallest element of a list without using built-in functions.

```
nums = [22, 43, 24, 55, 7]

largest = smallest = nums[0]
for num in nums:
    if num > largest:
        largest = num
    if num < smallest:
        smallest = num

print("Largest =", largest, " Smallest =", smallest)
print("Built-in:", max(nums), min(nums))
```

***Note:*** *Start from nums[0], not 0 - otherwise negative lists give a wrong answer.*

### Q92. Find the second largest element in a list.

```
nums = [22, 43, 24, 55, 7, 55]

first = second = float("-inf")
for num in nums:
    if num > first:
        second = first
        first = num
    elif first > num > second:
        second = num

print("Second largest =", second)
print("Using set   :", sorted(set(nums))[-2])
```

***Note:*** *Use set() first so duplicates of the largest value do not become the answer.*

### Q93. Find the sum and average of all elements in a list.

```
nums = [10, 20, 30, 40]

total = 0
for num in nums:
    total += num

print("Sum     =", total)
print("Average =", total / len(nums))
print("Built-in:", sum(nums), sum(nums) / len(nums))
```

***Note:*** *Guard against an empty list, otherwise len(nums) is 0 and you get ZeroDivisionError.*

### Q94. Print all even numbers and count positives / negatives in a list.

```
nums = [3, -4, 7, -2, 10, 0]

evens = [n for n in nums if n % 2 == 0]
print("Evens :", evens)

positives = len([n for n in nums if n > 0])
negatives = len([n for n in nums if n < 0])
print(f"Positive={positives} Negative={negatives}")
```

### Q95. Find the common elements between two lists.

```
list1 = [1, 23, 44, 4]
list2 = [2, 4, 23, 5]

common = list(set(list1) & set(list2))   # & means intersection
print(sorted(common))                    # [4, 23]

# without sets
print([x for x in list1 if x in list2])
```

***Note:*** *The & operator only works on sets, not on lists - convert first.*

### Q96. Merge two lists and remove duplicates.

```
list1 = [1, 2, 4, 6]
list2 = [4, 5, 2, 7]

merged = list1 + list2            # use +, not [list1 + list2]
print("Merged :", merged)
print("Unique :", sorted(set(merged)))
```

***Note:*** *[list1 + list2] wraps everything in one extra list - a common mistake.*

### Q97. Count the frequency of each element in a list.

```
nums = [1, 2, 3, 4, 2, 1, 4, 4]

freq = {}
for num in nums:
    freq[num] = freq.get(num, 0) + 1
print(freq)

from collections import Counter    # capital C
print(dict(Counter(nums)))
```

***Note:*** *The class name is Counter with a capital C, imported from collections.*

### Q98. Check whether a list is a palindrome.

```
list1 = [1, 4, 1]

if list1 == list1[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")

# careful: list.reverse() returns None, it changes the list itself
```

> ***Note:*** *list2 = list1.reverse() stores None. Use slicing [::-1] instead.*

# 9. Lists - Slicing, Copying and Comprehensions

### Q99. Take a list of numbers as input from the user in one line.

```
nums = list(map(int, input("Enter numbers separated by space: ").split()))
print(nums, "->", sum(nums))

# same thing with a comprehension
# nums = [int(x) for x in input().split()]
```

> ***Note:*** *split() breaks the line into strings, map(int, ...) converts each one to an integer.*

### Q100. Take 3 movie names from the user and store them in a list.

```
movies = []
for i in range(1, 4):
    movies.append(input(f"Enter movie {i}: "))

print(movies)
```

> ***Note:*** *Use a loop instead of writing append(input()) three times.*

### Q101. Show list slicing and how to replace a part of a list.

```
fruits = ["apple", "orange", "mango", "grapes", "kiwi"]

print(fruits[2])        # mango
print(fruits[1:3])      # ["orange", "mango"]
print(fruits[-1])       # kiwi
print(fruits[:2])       # first two

fruits[1:3] = ["banana", "papaya"]   # replace a slice
print(fruits)

del fruits[0]           # delete by index
print(fruits)
```

### Q102. Show the difference between copying a list and referencing it.

```
original = [1, 2, 3]

reference = original          # SAME list, two names
reference.append(4)
print(original)               # [1, 2, 3, 4]  -> changed!

copy_list = original.copy()   # or list(original) / original[:]
copy_list.append(5)
print(original)               # [1, 2, 3, 4]  -> safe
```

> ***Note:*** *Assignment does not copy a list, it only creates another name for the same object.*

### Q103. Show shallow copy vs deep copy.

```
import copy

nested = [[1, 2], [3, 4]]

shallow = copy.copy(nested)      # inner lists are still shared
shallow[0].append(99)
print(nested)                    # [[1, 2, 99], [3, 4]]

deep = copy.deepcopy(nested)     # fully independent
deep[0].append(100)
print(nested)                    # unchanged
```

***Note:*** *Shallow copy copies only the outer list. Deep copy copies every level. Asked very often.*

### Q104. Sort a list of tuples by the second element.

```
students = [("Surya", 78), ("Amit", 92), ("Riya", 65)]

students.sort(key=lambda item: item[1])
print(students)

print(sorted(students, key=lambda item: item[1], reverse=True))
```

***Note:*** *key tells sort() what value to compare. lambda item: item[1] means "use the marks".*

### Q105. Show the difference between sort() and sorted().

```
nums = [3, 1, 2]

new_list = sorted(nums)      # returns a NEW list
print(nums, new_list)        # [3, 1, 2] [1, 2, 3]

result = nums.sort()         # changes nums, returns None
print(nums, result)          # [1, 2, 3] None
```

***Note:*** *sort() works in place and returns None. sorted() returns a new sorted list.*

### Q106. Create a list of squares using a list comprehension.

```
squares = [x ** 2 for x in range(1, 6)]
print(squares)                    # [1, 4, 9, 16, 25]

even_squares = [x ** 2 for x in range(1, 11) if x % 2 == 0]
print(even_squares)               # [4, 16, 36, 64, 100]

labels = ["even" if x % 2 == 0 else "odd" for x in range(1, 5)]
print(labels)
```

***Note:*** *Format: [expression for item in iterable if condition].*

### Q107. Flatten a nested list into a single list.

```
nested = [[1, 2], [3, 4], [5, 6]]

flat = [item for sublist in nested for item in sublist]
print(flat)          # [1, 2, 3, 4, 5, 6]

# normal loop version
flat2 = []
for sublist in nested:
    for item in sublist:
        flat2.append(item)
```

***Note:*** *In a nested comprehension the loops are written in the same order as normal for loops.*

### Q108. Rotate a list to the left by k positions.

```
nums = [1, 2, 3, 4, 5]
k = 2

k = k % len(nums)                # handles k bigger than the list
rotated = nums[k:] + nums[:k]
print(rotated)                   # [3, 4, 5, 1, 2]
```

### Q109. Check whether a list is already sorted.

```
nums = [1, 2, 3, 5, 4]

is_sorted = True
for i in range(len(nums) - 1):
    if nums[i] > nums[i + 1]:
        is_sorted = False
        break

print("Sorted" if is_sorted else "Not sorted")
print(nums == sorted(nums))       # shortcut
```

***Note:*** *Loop only to len(nums) - 1, otherwise nums[i + 1] goes out of range.*

### Q110. Print a matrix (list of lists) row by row.

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

for row in matrix:
    for value in row:
        print(value, end=" ")
    print()

print("Element at row 1, column 2 :", matrix[1][2])   # 6
```

***Note:*** *matrix[row][column] - the first index is the row.*

# 10. Tuples, Sets and Dictionaries

### Q111. Create a tuple and use its methods.

```
my_tuple = ("apple", "orange", "mango", "apple")

print(my_tuple[0])                # apple
print(my_tuple[-1])               # apple
print(len(my_tuple))              # 4
print(my_tuple.count("apple"))    # 2
print(my_tuple.index("mango"))    # 2

# my_tuple[0] = "kiwi"   -> TypeError, tuples are immutable
```

***Note:*** *A tuple has only two methods, count() and index(), because it cannot be changed.*

### Q112. Change a tuple by converting it to a list.

```
my_tuple = (1, 2, 3)

temp = list(my_tuple)
temp.append(4)
my_tuple = tuple(temp)
print(my_tuple)          # (1, 2, 3, 4)

single = (5,)            # comma is needed for a 1-item tuple
print(type(single))
```

***Note:*** *(5) is just the number 5. (5,) with a comma is a tuple - a favourite trick question.*

### Q113. Unpack a tuple, including with a star (*).

```
person = ("Surya", 23, "Bhubaneswar")

name, age, city = person
print(name, age, city)

first, *rest = (1, 2, 3, 4, 5)
print(first, rest)          # 1 [2, 3, 4, 5]
```

***Note:*** *The number of variables must match, unless you use * to collect the extras.*

### Q114. Create a set and use its methods.

```
a = {1, 2, 3, 4, 5}

a.add(6)             # add one item
a.update([7, 8])     # add many items
a.remove(2)          # error if missing
a.discard(100)       # no error if missing
print(a)

print(len(a), 3 in a)
a.clear()
print(a)             # set()
```

***Note:*** *A set is unordered and cannot hold duplicates, so it has no indexing.*

### Q115. Perform union, intersection, difference and symmetric difference on two sets.

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7}

print(a.union(b))                  # a | b  -> all items
print(a.intersection(b))           # a & b  -> common items
print(a.difference(b))             # a - b  -> only in a
print(a.symmetric_difference(b))   # a ^ b  -> not common
```

> ***Note:*** *Remember the operator shortcuts: | & - ^*

### Q116. Remove duplicates from a list using a set and explain why order changes.

```
nums = [3, 1, 2, 3, 1]

print(set(nums))           # {1, 2, 3}  -> duplicates gone, order lost
print(sorted(set(nums)))   # [1, 2, 3]  -> order fixed by sorting
```

> ***Note:*** *Sets do not remember insertion order, so sort if you need a fixed order.*

### Q117. Create a dictionary and use its methods.

```
student = {"name": "Surya", "age": 23, "city": "Bhubaneswar"}

print(student["name"])            # error if key is missing
print(student.get("marks"))       # None if key is missing
print(student.get("marks", 0))    # 0 as a default value

print(student.keys())
print(student.values())
print(student.items())

student.update({"age": 24})       # update or add
student["course"] = "MCA"         # add a new key
student.pop("city")               # remove by key
print(student, len(student))
```

> ***Note:*** *Use get() when a key may be missing; [] raises KeyError.*

### Q118. Loop through a dictionary in three ways.

```
student = {"name": "Surya", "age": 23, "course": "MCA"}

for key in student:                    # keys only
    print(key, "->", student[key])

for value in student.values():         # values only
    print(value)

for key, value in student.items():     # both together (best)
    print(key, ":", value)
```

> ***Note:*** *items() gives key and value in one step - the cleanest way.*

### Q119. Store the marks of several students in a nested dictionary.

```
students = {}

for _ in range(2):
    name = input("Enter name: ")
    phy = int(input("Physics marks: "))
    maths = int(input("Maths marks: "))
    students[name] = {"phy": phy, "maths": maths}

print(students)
for name, marks in students.items():
    print(name, "total =", marks["phy"] + marks["maths"])
```

***Note:*** *A dictionary value can itself be a dictionary - this is called nesting.*

### Q120. Merge two dictionaries.

```
a = {"name": "Surya", "age": 23}
b = {"age": 24, "city": "Bhubaneswar"}

merged = {**a, **b}          # works in Python 3.5+
print(merged)                # age becomes 24 (b wins)

a.update(b)                  # changes a itself
print(a)
```

***Note:*** *When the same key exists in both, the value from the second dictionary wins.*

### Q121. Sort a dictionary by its values.

```
marks = {"Surya": 78, "Amit": 92, "Riya": 65}

sorted_marks = dict(sorted(marks.items(), key=lambda item: item[1], reverse=True))
print(sorted_marks)

print("Topper :", max(marks, key=marks.get))
```

***Note:*** *max(dict, key=dict.get) returns the key that has the biggest value.*

### Q122. Create a dictionary and a set using comprehensions, and invert a dictionary.

```
squares = {x: x ** 2 for x in range(1, 6)}
print(squares)                  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

unique_lengths = {len(w) for w in ["hi", "bye", "ok"]}
print(unique_lengths)           # {2, 3}

original = {"a": 1, "b": 2}
inverted = {v: k for k, v in original.items()}
print(inverted)                 # {1: "a", 2: "b"}
```

***Note:*** *Curly braces with key: value make a dict comprehension; without the colon it is a set.*

# 11. Functions - Arguments and Return Values

### Q123. Write a function that prints a message and call it.

```
def my_func():
    print("I am Surya")

my_func()
my_func()      # can be called any number of times
```

***Note:*** *def defines the function; the code runs only when you call it with ().*

### Q124. Write a function that returns a value, and show the difference from print.

```
def get_name():
    return "I am Surya"      # sends the value back

message = get_name()         # value is stored
print(message)

def show_name():
    print("I am Surya")      # only displays, returns None

print(show_name())           # I am Surya  then  None
```

***Note:*** *return gives a value back to the caller; print only shows it on the screen.*

### Q125. Write a function that takes two numbers and returns their sum.

```
def calc_sum(a, b):
    return a + b

result = calc_sum(22, 33)
print(result)                # 55
```

***Note:*** *a and b are parameters; 22 and 33 are arguments.*

### Q126. Write a function with a default parameter value.

```
def greet(name="friend"):
    print("Hello", name)

greet("Surya")     # Hello Surya
greet()            # Hello friend  -> default used
```

***Note:*** *Parameters with default values must come after the ones without defaults.*

### Q127. Call a function using keyword arguments.

```
def pet_info(animal, name):
    print("My pet is a", animal)
    print("Its name is", name)

pet_info(animal="dog", name="Seru")    # order does not matter
pet_info("cat", "Kitty")               # positional call
```

***Note:*** *Keyword arguments make the call readable and order-independent.*

### Q128. Pass a list and a dictionary to a function.

```
def show_fruits(fruits):
    for fruit in fruits:          # loop over the list
        print(fruit)

show_fruits(["apple", "banana", "cherry"])

def show_person(person):
    print("Name:", person["name"])
    print("Age :", person["age"])

show_person({"name": "Surya", "age": 23})
```

***Note:*** *Inside the loop print fruit (the item), not fruits (the whole list).*

### Q129. Return multiple values from a function.

```
def calc(a, b):
    return a + b, a - b, a * b     # returns a tuple

add, sub, mul = calc(10, 3)
print(add, sub, mul)               # 13 7 30
```

***Note:*** *Python packs multiple return values into a tuple, which you can then unpack.*

### Q130. Write a function that accepts any number of arguments using *args.

```
def total_sum(*numbers):
    total = 0
    for num in numbers:            # numbers is a tuple
        total += num
    return total

print(total_sum(1, 2, 3))          # 6
print(total_sum(10, 20, 30, 40))   # 100
print(total_sum())                 # 0
```

***Note:*** **args collects extra positional arguments into a tuple.*

### Q131. Find the maximum value using *args.

```
def find_max(*numbers):
    if len(numbers) == 0:
        return None
    max_num = numbers[0]
    for num in numbers:
        if num > max_num:
            max_num = num
    return max_num

print(find_max(3, 7, 2, 9, 1))     # 9
```

***Note:*** *Always handle the empty case before touching numbers[0].*

### Q132. Write a function that accepts any number of keyword arguments using **kwargs.

```
def show_details(**details):
    print("Name :", details["name"])
    print("Age  :", details["age"])
    print("All  :", details)       # details is a dictionary

show_details(name="Surya", age=23, city="Nayagarh")
```

***Note:*** ***kwargs collects extra keyword arguments into a dictionary.*

### Q133. Unpack a list and a dictionary while calling a function.

```
def add(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
print(add(*numbers))               # 6   -> unpacks the list

def full_name(fname, lname):
    print("Hello", fname, lname)

name = {"fname": "Surya", "lname": "Pradhan"}
full_name(**name)                  # unpacks the dictionary
```

***Note:*** ** unpacks a sequence into positional arguments, ** unpacks a dict into keyword arguments.*

### Q134. Show positional-only and keyword-only parameters ( / and * ).

```
def my_func(a, b, /, *, c, d):
    return a + b + c + d

print(my_func(5, 2, c=10, d=22))   # 39

# my_func(a=5, b=2, c=10, d=22)  -> error, a and b must be positional
# my_func(5, 2, 10, 22)          -> error, c and d must be keywords
```

***Note:*** *Everything before / must be positional; everything after * must be a keyword.*

### Q135. Show local scope, global scope and the global keyword.

```
x = "global surya"

def show_local():
    x = "local kumar"        # a different, local variable
    print(x)

show_local()                 # local kumar
print(x)                     # global surya

def change_global():
    global x                 # now it points to the outer x
    x = "changed"

change_global()
print(x)                     # changed
```

***Note:*** *Without global, assigning inside a function creates a new local variable.*

### Q136. Show the nonlocal keyword with a nested function.

```
def outer():
    x = "surya"

    def inner():
        nonlocal x           # use the outer function's x
        x = "kanta"

    inner()
    return x

print(outer())               # kanta
```

***Note:*** *global reaches the module level; nonlocal reaches the enclosing function.*

### Q137. Show the mutable default argument trap.

```
def wrong(item, box=[]):        # BAD: the list is created only once
    box.append(item)
    return box

print(wrong(1))                 # [1]
print(wrong(2))                 # [1, 2]  -> surprise!

def right(item, box=None):      # GOOD
    if box is None:
        box = []
    box.append(item)
    return box

print(right(1), right(2))       # [1] [2]
```

***Note:*** *Never use a list or dict as a default value. Use None and create it inside.*

# 12. Lambda, Decorators, Generators and Recursion

### Q138. Write a lambda function to multiply two numbers.

```
multiply = lambda a, b: a * b
print(multiply(5, 6))          # 30

square = lambda x: x ** 2
print(square(4))               # 16
```

***Note:*** *A lambda is a small one-line function with no name and no return keyword.*

### Q139. Return a lambda from a function (closure) to make a doubler and a tripler.

```
def multiplier(n):
    return lambda a: a * n     # remembers n

doubler = multiplier(2)
tripler = multiplier(3)

print(doubler(5))              # 10
print(tripler(5))              # 15
```

***Note:*** *A function that remembers a value from the outer function is called a closure.*

### Q140. Use map(), filter() and reduce() with lambda.

```
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]

doubled = list(map(lambda x: x * 2, numbers))
print(doubled)                 # [2, 4, 6, 8, 10, 12]

odds = list(filter(lambda x: x % 2 != 0, numbers))
print(odds)                    # [1, 3, 5]

total = reduce(lambda a, b: a + b, numbers)
print(total)                   # 21
```

***Note:*** *map transforms every item, filter keeps some items, reduce combines all into one value.*

### Q141. Use zip(), enumerate(), any() and all().

```
names = ["Surya", "Amit", "Riya"]
marks = [78, 92, 65]

for name, mark in zip(names, marks):
    print(name, mark)

for i, name in enumerate(names, start=1):
    print(i, name)

print(any(m > 90 for m in marks))    # True  -> at least one
print(all(m > 60 for m in marks))    # True  -> every one
```

***Note:*** *zip() pairs up lists and stops at the shorter one.*

### Q142. Write a decorator that runs code before and after a function.

```
def my_decorator(func):
    def wrapper():
        print("Before the function")
        func()
        print("After the function")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

> ***Note:*** *A decorator takes a function, wraps it with extra behaviour and returns the new function.*

### Q143. Write a decorator that works with any arguments and converts the result to uppercase.

```
def to_upper(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

@to_upper
def greet(name):
    return "hello " + name

print(greet("surya"))          # HELLO SURYA
```

> ***Note:*** *Use *args, **kwargs in the wrapper so the decorator fits any function.*

### Q144. Write a generator function using yield.

```
def my_generator():
    yield 1
    yield 2
    yield 3

for value in my_generator():
    print(value)

gen = my_generator()
print(next(gen))               # 1
print(next(gen))               # 2
```

> ***Note:*** *yield pauses the function and remembers where it stopped. return would end it.*

### Q145. Write a generator that counts up to N, and explain why it saves memory.

```
def count_up_to(n):
    count = 1
    while count <= n:          # condition must use n
        yield count
        count += 1

for num in count_up_to(5):
    print(num, end=" ")

# a generator makes values one by one, so 10 lakh numbers
# do not have to sit in memory together
```

***Note:*** *Generators are lazy: they produce one value at a time instead of building a full list.*

### Q146. Show the difference between a list comprehension and a generator expression.

```
squares_list = [x ** 2 for x in range(5)]    # list, uses memory now
squares_gen = (x ** 2 for x in range(5))    # generator, lazy

print(squares_list)            # [0, 1, 4, 9, 16]
print(squares_gen)             # <generator object ...>
print(list(squares_gen))       # [0, 1, 4, 9, 16]
print(list(squares_gen))       # []  -> already used up
```

***Note:*** *Square brackets build a list; round brackets build a generator that can be used only once.*

### Q147. Write a custom iterator class using __iter__ and __next__.

```
class MyNumbers:
    def __iter__(self):
        self.a = 1
        return self

    def __next__(self):
        if self.a <= 5:
            x = self.a
            self.a += 1
            return x
        raise StopIteration        # capital S, capital I

for num in MyNumbers():
    print(num, end=" ")
```

***Note:*** *Without raise StopIteration the for loop never ends.*

### Q148. Find the factorial of a number using recursion.

```
def factorial(n):
    if n == 0 or n == 1:       # base case - stops the recursion
        return 1
    return n * factorial(n - 1)

print(factorial(5))            # 120
```

***Note:*** *Every recursive function needs a base case, otherwise you get RecursionError.*

### Q149. Print the Fibonacci series using recursion.

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(7):
    print(fibonacci(i), end=" ")   # 0 1 1 2 3 5 8
```

> ***Note:*** *Simple but slow - it recalculates the same values many times (exponential time).*

### Q150. Find the sum of a list and the sum of 1 to N using recursion.

```
def list_sum(numbers):
    if len(numbers) == 0:
        return 0
    return numbers[0] + list_sum(numbers[1:])

print(list_sum([1, 2, 3, 4, 5]))      # 15

def n_sum(n):
    if n == 0:
        return 0
    return n + n_sum(n - 1)

print(n_sum(5))                        # 15
```

### Q151. Write a recursive countdown and find the maximum of a list recursively.

```
def countdown(n):
    if n <= 0:
        print("Done")
        return
    print(n, end=" ")
    countdown(n - 1)

countdown(5)                           # 5 4 3 2 1 Done

def find_max(numbers):
    if len(numbers) == 1:
        return numbers[0]
    return max(numbers[0], find_max(numbers[1:]))

print(find_max([3, 7, 2, 9, 5]))       # 9
```

### Q152. Print a name N times using recursion with a counter.

```
def print_name(count, n):
    if count > n:
        return
    print("Surya", end=" ")
    print_name(count + 1, n)

print_name(1, 5)

# passing the counter as a parameter is cleaner than using a global
```

> ***Note:*** *Prefer parameters over global variables in recursion - easier to test and reason about.*

# 13. Exception Handling and File Handling

### Q153. Handle an error using try and except.

```
try:
    num = int(input("Enter a number: "))
    print("Result =", 100 / num)
except ValueError:
    print("Please enter digits only")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

***Note:*** *Catch specific exceptions. A bare except: hides real bugs.*

### Q154. Use try / except / else / finally together.

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Invalid input")
else:
    print("No error, the number is", num)   # runs only if no error
finally:
    print("This always runs")               # cleanup code
```

***Note:*** *else runs when there was no exception; finally always runs, error or not.*

### Q155. Catch more than one exception in a single except block.

```
try:
    values = [1, 2, 3]
    print(values[5])
except (IndexError, KeyError) as e:
    print("Error type :", type(e).__name__)
    print("Message    :", e)
```

***Note:*** *as e gives you the exception object, useful for logging the message.*

### Q156. Raise an exception yourself.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    print("Age set to", age)

try:
    set_age(-5)
except ValueError as e:
    print("Caught:", e)
```

***Note:*** *raise creates an error on purpose - used to validate input.*

### Q157. Create and use a custom exception class.

```
class InsufficientBalance(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalance("Not enough money")
    return balance - amount

try:
    print(withdraw(500, 1000))
except InsufficientBalance as e:
    print("Error:", e)
```

***Note:*** *A custom exception is just a class that inherits from Exception.*

### Q158. Write text to a file.

```
with open("data.txt", "w") as f:      # "w" overwrites the file
    f.write("Hello Surya\n")
    f.write("Learning file handling\n")

with open("data.txt", "a") as f:      # "a" adds at the end
    f.write("This line is appended\n")

print("File written successfully")
```

***Note:*** *"w" deletes the old content, "a" keeps it and appends.*

### Q159. Read a whole file and read it line by line.

```
with open("data.txt", "r") as f:
    print(f.read())               # whole file as one string

with open("data.txt", "r") as f:
    for line in f:                # memory friendly for big files
        print(line.strip())

with open("data.txt", "r") as f:
    lines = f.readlines()         # list of lines
print(len(lines), "lines")
```

***Note:*** *Reading line by line is better for large files than read() which loads everything.*

### Q160. Explain why the with statement is used for files.

```
# without with - you must close it yourself
f = open("data.txt", "r")
content = f.read()
f.close()                         # easy to forget!

# with - closes the file automatically, even if an error happens
with open("data.txt", "r") as f:
    content = f.read()
print("File closed?", f.closed)   # True
```

***Note:*** *with is a context manager: it guarantees the file is closed.*

### Q161. Handle a missing file safely.

```
try:
    with open("no_such_file.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("File does not exist")
except PermissionError:
    print("No permission to read this file")
```

> ***Note:*** *File modes: r read, w write (overwrite), a append, r+ read and write, rb/wb binary.*

### Q162. Count the words and lines in a file.

```
with open("data.txt", "r") as f:
    lines = f.readlines()

word_count = 0
for line in lines:
    word_count += len(line.split())

print("Lines =", len(lines))
print("Words =", word_count)
```

### Q163. Read a number from one file and write the multiplication table to another file.

```
with open("input.txt", "w") as f:     # create sample input
    f.write("5")

with open("input.txt", "r") as f:
    n = int(f.read().strip())

result = ""
for i in range(1, 11):
    result += f"{n} x {i} = {n * i}\n"

with open("output.txt", "w") as f:
    f.write(result)

print("Table written to output.txt")
```

> ***Note:*** *strip() removes the trailing newline before int() - otherwise it can crash.*

# 14. Object Oriented Programming (Basics)

### Q164. Create a class with __init__ and make an object.

```
class Student:
    def __init__(self, name, roll):    # constructor
        self.name = name                # instance variables
        self.roll = roll

    def show(self):
        print(f"Name: {self.name}, Roll: {self.roll}")

s1 = Student("Surya", 383)
s2 = Student("Amit", 384)
s1.show()
s2.show()
```

> ***Note:*** *__init__ runs automatically when an object is created. self refers to that object.*

### Q165. Show the difference between a class variable and an instance variable.

```
class Student:
    college = "KIIT"               # class variable - shared

    def __init__(self, name):
        self.name = name           # instance variable - separate

s1 = Student("Surya")
s2 = Student("Amit")

print(s1.college, s2.college)      # KIIT KIIT
Student.college = "Utkal"          # changes it for everyone
print(s1.college, s2.college)      # Utkal Utkal
```

> ***Note:*** *A class variable belongs to the class; an instance variable belongs to one object.*

### Q166. Write a class with a method that returns a calculated value.

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r = Rectangle(5, 3)
print("Area =", r.area())
print("Perimeter =", r.perimeter())
```

> ***Note:*** *Every method needs self as the first parameter to reach the object's data.*

### Q167. Show single inheritance.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(self.name, "makes a sound")

class Dog(Animal):                # Dog inherits from Animal
    def fetch(self):
        print(self.name, "fetches the ball")

d = Dog("Seru")
d.speak()      # from the parent class
d.fetch()      # its own method
```

***Note:*** *Inheritance lets a child class reuse the parent's code. Types: single, multilevel, multiple, hierarchical, hybrid.*

### Q168. Show method overriding and super().

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Some sound")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)     # call the parent constructor
        self.breed = breed

    def speak(self):               # overrides the parent method
        print(self.name, "says Bhow Bhow")

d = Dog("Seru", "Indie")
d.speak()
print(d.breed)
```

***Note:*** *Overriding = the child defines a method with the same name. super() calls the parent version.*

### Q169. Show polymorphism with the same method name in different classes.

```
class Dog:
    def speak(self):
        print("Bhow Bhow")

class Cat:
    def speak(self):
        print("Meow")

for animal in [Dog(), Cat()]:      # same call, different behaviour
    animal.speak()
```

***Note:*** *Python does not check the type, only whether the method exists - this is duck typing.*

### Q170. Show encapsulation with private variables.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance       # __ makes it private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

a = Account(1000)
a.deposit(500)
print(a.get_balance())               # 1500
# print(a.__balance)  -> AttributeError
```

***Note:*** *Double underscore triggers name mangling. It hides data but is not true security.*

### Q171. Show __str__ and __repr__ (dunder methods).

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __str__(self):                 # for the user
        return f"{self.title} - Rs.{self.price}"

    def __repr__(self):                # for the developer
        return f"Book({self.title!r}, {self.price})"

b = Book("Python", 350)
print(b)          # uses __str__
print(repr(b))    # uses __repr__
```

***Note:*** *Without __str__, print(object) shows an ugly memory address.*

### Q172. Show static methods and class methods.

```
class Maths:
    count = 0

    def __init__(self):
        Maths.count += 1

    @staticmethod
    def add(a, b):                     # no self, no cls
        return a + b

    @classmethod
    def total_objects(cls):            # gets the class as cls
        return cls.count

Maths(); Maths()
print(Maths.add(2, 3))                 # 5
print(Maths.total_objects())           # 2
```

***Note:*** *staticmethod is a plain helper; classmethod works on the class itself.*

### Q173. Count how many objects of a class have been created.

```
class Student:
    count = 0

    def __init__(self, name):
        self.name = name
        Student.count += 1            # use the class name

Student("Surya")
Student("Amit")
Student("Riya")
print("Total students =", Student.count)   # 3
```

***Note:*** *Write Student.count, not self.count, otherwise each object gets its own copy.*

### Q174. Show the difference between == and is.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)      # True  -> same values
print(a is b)      # False -> different objects in memory
print(a is c)      # True  -> same object

print(id(a), id(b), id(c))
```

***Note:*** *== compares values, is compares identity (memory address). Use is only with None.*

### Q175. Create a simple bank account class with deposit, withdraw and a menu.

```
class Bank:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited", amount)

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance")
        else:
            self.balance -= amount
            print("Withdrawn", amount)

acc = Bank("Surya", 1000)
acc.deposit(500)
acc.withdraw(2000)
print("Balance =", acc.balance)
```

***Note:*** *A classic interview task - it tests classes, methods and validation together.*

# 15. Modules and Practical Extras

### Q176. Import a module in different ways and use it.

```
import math
print(math.sqrt(16), math.pi, math.ceil(4.1), math.floor(4.9))

from math import sqrt, pow
print(sqrt(25), pow(2, 3))

import math as m               # alias
print(m.factorial(5))
```

***Note:*** *import module keeps the namespace; from module import name brings it in directly.*

### Q177. Use the random module.

```
import random

print(random.randint(1, 10))             # 1 to 10, both included
print(random.random())                   # 0.0 to 1.0
print(random.choice(["apple", "mango"]))  # random item

nums = [1, 2, 3, 4, 5]
random.shuffle(nums)                     # shuffles in place
print(nums)
```

***Note:*** *randint(a, b) includes both ends, unlike range(a, b).*

### Q178. Print the current date and time using the datetime module.

```
from datetime import datetime, date, timedelta

now = datetime.now()
print("Now      :", now)
print("Formatted:", now.strftime("%d-%m-%Y %H:%M:%S"))
print("Today    :", date.today())
print("Tomorrow :", date.today() + timedelta(days=1))
```

***Note:*** *strftime() turns a date into text; strptime() turns text into a date.*

### Q179. Explain and use if __name__ == "__main__".

```
def main():
    print("Program started")

if __name__ == "__main__":
    main()      # runs only when this file is run directly

# if this file is imported into another file,
# __name__ becomes the module name and main() does not run
```

***Note:*** *This guard stops your test code from running when the file is imported.*

### Q180. Write a program that keeps taking numbers until the user types "done", then shows a summary.

```
numbers = []

while True:
    value = input("Enter a number (or done): ")
    if value.lower() == "done":
        break
    try:
        numbers.append(float(value))
    except ValueError:
        print("Not a number, try again")

if numbers:
    print(f"Count={len(numbers)} Sum={sum(numbers)} "
          f"Max={max(numbers)} Avg={sum(numbers) / len(numbers):.2f}")
else:
    print("No numbers entered")
```

***Note:*** *Combines loops, break, exception handling and built-in functions - a good practice task.*

### Q181. Show truthy and falsy values in Python.

```
values = [0, 1, -1, "", "a", [], [0], {}, None, 0.0]

for v in values:
    print(repr(v), "->", bool(v))

# Falsy: 0, 0.0, "", [], (), {}, set(), None, False
# Everything else is truthy
```

***Note:*** *This is why if my_list: is the correct way to check for an empty list.*

# PART B - INTERVIEW THEORY QUESTIONS

Answer these in your own words in 2-3 sentences. Where a difference is asked, always give one clear example - that is what separates a good answer from a textbook answer.

# A. Language Basics and Data Types

### Q182. What is Python and what are its main features?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum. Main features: simple English-like syntax, dynamically typed (no need to declare types), interpreted (runs line by line), platform independent, object oriented, free and open source, and it has a very large standard library plus packages like NumPy, Pandas and Django.

### Q183. Is Python compiled or interpreted?

Both, in stages. Python source code is first compiled into bytecode (.pyc files inside __pycache__), and that bytecode is then executed line by line by the Python Virtual Machine (PVM). Because we do not run a separate compile step, we normally call Python an interpreted language.

### Q184. What does it mean that Python is dynamically typed?

You do not declare the type of a variable; the type is decided at runtime from the value. x = 10 makes x an int, and then x = "hello" makes the same name a str. Dynamically typed does not mean type-free: Python still raises TypeError for invalid operations like "a" + 1.

### Q185. What are the built-in data types in Python?

Numeric: int, float, complex. Sequence: str, list, tuple, range. Set types: set, frozenset. Mapping: dict. Boolean: bool. Binary: bytes, bytearray, memoryview. And NoneType for None.

### Q186. What is the difference between mutable and immutable objects?

Mutable objects can be changed after creation: list, dict, set, bytearray. Immutable objects cannot be changed: int, float, str, tuple, bool, frozenset. With an immutable object any "change" actually creates a new object, which is why s = s + "a" gives a new string.

### Q187. What is the difference between a list and a tuple?

A list is mutable, written with [], and has many methods (append, remove, sort). A tuple is immutable, written with (), and has only count() and index(). Tuples are slightly faster and use less memory, and because they are immutable they can be used as dictionary keys - a list cannot.

### Q188. What is the difference between a list, a set and a dictionary?

A list is an ordered collection that allows duplicates and is accessed by index. A set is an unordered collection with no duplicates and no indexing, used for membership tests and removing duplicates. A dictionary stores key-value pairs, keys must be unique and immutable, and lookup is by key. Set and dict lookups are O(1) on average; searching a list is O(n).

### Q189. What is the difference between == and is?

== compares the values of two objects. is compares identity, i.e. whether both names point to the same object in memory. Two lists with the same items are == but not is. The is operator should mainly be used with None (if x is None).

### Q190. What is the difference between shallow copy and deep copy?

A shallow copy (copy.copy(), list.copy(), slicing) copies only the outer object; nested objects are still shared, so changing an inner list affects both copies. A deep copy (copy.deepcopy()) recursively copies every level, so the two objects are fully independent.

### Q191. What is the difference between /, // and % ?

/ is true division and always returns a float (7 / 2 = 3.5). // is floor division and returns the quotient without the decimal part (7 // 2 = 3). % is modulus and returns the remainder (7 % 2 = 1). ** is the power operator (2 ** 3 = 8).

### Q192. What is slicing and how does negative indexing work?

Slicing takes a part of a sequence using [start:stop:step]. The stop index is never included. Negative indexes count from the end: s[-1] is the last item, s[-2] the second last. s[::-1] reverses the sequence, and s[:] makes a shallow copy.

### Q193. Why are strings immutable in Python, and what does that mean in practice?

A string object cannot be modified after creation, so s[0] = "S" raises TypeError. Every operation that looks like a change (upper(), replace(), concatenation) returns a new string. In practice, building a big string in a loop with += is slow; collect the parts in a list and use "".join(parts) instead.

### Q194. Are dictionaries ordered in Python?

From Python 3.7 onwards a normal dict keeps the insertion order as a language guarantee (it was already true in CPython 3.6). Before that, order was not guaranteed and people used collections.OrderedDict. Sets are still unordered.

### Q195. What are truthy and falsy values?

Any value can be used in a condition. Falsy values are 0, 0.0, empty string "", empty list [], empty tuple (), empty dict {}, empty set, None and False. Everything else is truthy. This is why if my_list: is the preferred way to test for a non-empty list.

### Q196. What is type conversion (casting) in Python?

Converting a value from one type to another. Implicit conversion happens automatically (int + float gives float). Explicit conversion is done by us with int(), float(), str(), list(), tuple(), set(), dict(). Since input() always returns a string, int(input()) is needed for numeric input.

### Q197. What is PEP 8 and why does it matter?

PEP 8 is the official style guide for Python code: 4 spaces for indentation, snake_case for variables and functions, PascalCase for class names, UPPER_CASE for constants, lines up to 79 characters, and blank lines between functions. Interviewers notice clean, consistent code even in a small program.

## B. Loops, Functions and Advanced Concepts

**Q198. What is the difference between break, continue and pass?**

break stops the whole loop immediately. continue skips the rest of the current iteration and goes to the next one. pass does nothing at all - it is only a placeholder used where Python expects a statement, for example in an empty function or class body.

**Q199. What is the use of else with a for or while loop?**

The loop's else block runs when the loop finishes normally, i.e. without hitting break. It is very useful in prime-number checks: if no divisor is found and break never runs, the else block declares the number prime.

**Q200. What is the difference between range() in Python 2 and Python 3?**

In Python 2 range() built a full list in memory and xrange() returned a lazy iterator. In Python 3, xrange() is gone and range() itself is lazy - it produces values on demand, so range(10**8) uses almost no memory. Wrap it in list() if you actually need a list.

**Q201. What is the difference between arguments and parameters?**

Parameters are the names written in the function definition, for example def add(a, b). Arguments are the actual values passed while calling, for example add(2, 3). Python supports positional, keyword, default and variable-length (*args, **kwargs) arguments.

**Q202. What is the difference between *args and **kwargs?**

*args collects any number of extra positional arguments into a tuple. **kwargs collects any number of extra keyword arguments into a dictionary. The correct order in a definition is: normal parameters, *args, default parameters, then **kwargs.

**Q203. What is the difference between local, global and nonlocal variables?**

A local variable is created inside a function and exists only there. A global variable is defined at module level and is readable everywhere; to assign to it inside a function you need the global keyword. nonlocal is used in a nested function to modify a variable of the enclosing function, not the module.

**Q204. What is a docstring?**

A string written as the first statement of a module, function or class, normally in triple quotes, that documents what it does. It is stored in the object's __doc__ attribute and is shown by help(). Unlike a # comment, a docstring stays available at runtime.

**Q205. What is a lambda function and what are its limitations?**

A lambda is a small anonymous function written in one line: square = lambda x: x ** 2. It can contain only a single expression, which is returned automatically. It cannot hold multiple statements, loops, or a return keyword, so for anything longer a normal def is better. Lambdas are mostly used as the key= argument in sort(), or with map() and filter().

### Q206. What is the difference between map(), filter() and reduce()?

map(func, iterable) applies the function to every item and returns all results. filter(func, iterable) returns only the items for which the function returns True. reduce(func, iterable) from functools combines all items into one single value, for example a total. map and filter return lazy iterators in Python 3, so wrap them in list().

### Q207. What is a list comprehension and how is it different from a generator expression?

A list comprehension [x*x for x in range(5)] builds the complete list in memory immediately. A generator expression (x*x for x in range(5)) creates a lazy generator that produces one value at a time and can be iterated only once. For large data the generator saves a lot of memory.

### Q208. What is a generator and how is yield different from return?

A generator is a function that uses yield to produce a series of values lazily. return ends the function and sends back one value, whereas yield pauses the function, sends a value out, and remembers the state so execution continues from that point on the next call. Generators are memory efficient because they never build the whole sequence.

### Q209. What is the difference between an iterable and an iterator?

An iterable is anything you can loop over (list, tuple, string, dict) - it implements __iter__(). An iterator is the object that actually does the walking - it implements both __iter__() and __next__() and raises StopIteration when it is finished. iter(my_list) turns an iterable into an iterator.

### Q210. What is a decorator?

A decorator is a function that takes another function as input, adds extra behaviour around it, and returns the new function. It is applied with the @decorator_name syntax written above the function. Common real uses are logging, timing, authentication checks and caching, and @staticmethod and @property are built-in examples.

### Q211. What is a closure?

A closure is an inner function that remembers variables from its enclosing function even after that outer function has finished. def multiplier(n): return lambda x: x * n - the returned lambda keeps n alive. Closures are the mechanism decorators are built on.

### Q212. What is recursion, and what are its advantages and disadvantages?

Recursion is a function calling itself until a base case stops it. Advantage: code becomes short and natural for problems like factorial, Fibonacci, and tree or folder traversal. Disadvantage: every call uses stack memory, so deep recursion is slow and can raise RecursionError - Python's default limit is about 1000 calls.

# C. OOP, Errors, Files and Memory

### Q213. What are the four pillars of OOP in Python?

Encapsulation - keeping data and methods together and hiding internal details (private variables with __). Inheritance - a child class reusing a parent class's code. Polymorphism - the same method name behaving differently for different classes. Abstraction - exposing only what is needed, using abstract base classes from the abc module.

### Q214. What is self and what is __init__?

self is a reference to the current object, and it must be the first parameter of every instance method so the method can reach that object's data. __init__ is the constructor: it runs automatically when the object is created and is used to set the initial values of the instance variables.

### Q215. What is the difference between a class variable and an instance variable?

A class variable is written directly inside the class body and is shared by all objects, so changing it through the class changes it everywhere. An instance variable is created with self.name inside a method and belongs to one object only. Use a class variable for shared data like a counter or a college name.

### Q216. What are the types of inheritance in Python?

Single (one parent), Multilevel (A to B to C), Multiple (a class inheriting from two or more parents), Hierarchical (many children from one parent) and Hybrid (a mix). With multiple inheritance Python decides which method to use via the MRO (Method Resolution Order), which you can see with ClassName.__mro__.

### Q217. What is method overriding, and does Python support method overloading?

Overriding is when a child class defines a method with the same name as the parent's, replacing its behaviour; super() calls the parent version. Python does not support true method overloading - if you define two methods with the same name, the last one wins. The same effect is achieved with default parameters or *args.

### Q218. What is the use of super()?

super() gives access to the parent class from inside a child class, without hard-coding the parent's name. It is mostly used as super().__init__(...) so the parent's constructor also runs, and to call the parent version of an overridden method.

### Q219. What is the difference between __str__ and __repr__?

__str__ returns a friendly string for the end user and is used by print() and str(). __repr__ returns an unambiguous developer-facing string, ideally one that could recreate the object, and is used by repr() and in the interactive shell. If only __repr__ is defined, print() will fall back to it.

### Q220. What is the difference between an instance method, a static method and a class method?

An instance method takes self and works on one object's data. A class method is decorated with @classmethod, takes cls, and works on class-level data (often used as an alternative constructor). A static method is decorated with @staticmethod, takes neither self nor cls, and is just a helper function kept inside the class for organisation.

### Q221. How does encapsulation work in Python? What is name mangling?

A single underscore (_name) is a convention meaning "internal, please do not touch". A double underscore (__name) triggers name mangling: Python renames it to _ClassName__name so it cannot be accessed accidentally from outside or by a subclass. It is protection by convention, not real security - the mangled name is still reachable.

### Q222. What is duck typing?

Python cares about what an object can do, not what it is. If an object has a speak() method, it can be used wherever speak() is called, regardless of its class - "if it walks like a duck and quacks like a duck, treat it as a duck". This is how Python achieves polymorphism without interfaces.

### Q223. What is the difference between an error and an exception?

A syntax error (or other compile-time error) means the code cannot even be parsed and nothing runs. An exception is a runtime problem in otherwise valid code - ZeroDivisionError, ValueError, TypeError, IndexError, KeyError, FileNotFoundError - and it can be caught and handled with try/except.

### Q224. Explain try, except, else and finally.

try holds the risky code. except catches a matching exception and handles it. else runs only when no exception occurred. finally always runs, whether there was an exception or not, and is used for cleanup such as closing files or database connections. Catch specific exceptions rather than using a bare except.

### Q225. What is the difference between raise and assert?

raise deliberately throws an exception, and is the normal way to signal invalid input in production code: raise ValueError("Age cannot be negative"). assert checks a condition and raises AssertionError if it is False; it is meant for debugging and internal checks, and it is removed when Python runs with the -O flag, so never use assert for input validation.

### Q226. What are the file modes, and why should we use the with statement?

Modes: "r" read (default, error if missing), "w" write (creates or overwrites), "a" append, "r+" read and write, and adding "b" (rb, wb) for binary files like images. The with statement is a context manager that closes the file automatically, even if an exception happens in the middle - so f.close() can never be forgotten.

### Q227. What is the GIL (Global Interpreter Lock)?

The GIL is a lock in CPython that allows only one thread to execute Python bytecode at a time. So threads do not give true parallel CPU usage, though they still help for I/O-bound work like file or network operations. For CPU-bound work use the multiprocessing module, which runs separate processes each with their own GIL.

### Q228. How does Python manage memory and garbage collection?

Python keeps objects in a private heap and manages it automatically. Each object has a reference count; when the count drops to zero the memory is released. A cyclic garbage collector (the gc module) additionally cleans up objects that reference each other. Small integers and short strings are cached and reused, which is why 'is' can behave surprisingly with small numbers.

### Q229. What is the difference between a module and a package? What is pip and a virtual environment?

A module is a single .py file you can import. A package is a folder of modules (traditionally containing an __init__.py). pip is the package installer used as pip install requests, and a virtual environment created with python -m venv env keeps each project's packages separate so different projects can use different versions.

## D. Rapid-fire Differences and Output Questions

**Q230. sort() vs sorted() - what is the difference?**

list.sort() sorts the list in place and returns None, and it works only on lists. sorted(iterable) returns a new sorted list and works on any iterable including strings, tuples and dictionaries. The classic bug is writing new = my_list.sort(), which stores None.

**Q231. append() vs extend() vs insert() - what is the difference?**

append(x) adds x as a single item at the end, so append([4,5]) creates a nested list. extend([4,5]) adds each element separately, giving a flat list. insert(i, x) adds x at index i and shifts the remaining items to the right.

**Q232. remove() vs pop() vs del vs clear() - what is the difference?**

remove(value) deletes the first matching value and raises ValueError if it is absent. pop(index) deletes by index and returns the removed item (pop() with no argument removes the last). del my_list[i] deletes by index without returning anything and can also delete slices or the whole variable. clear() empties the list.

**Q233. What is the output of print(10 / 3, 10 // 3, 10 % 3, 10 ** 3) ?**

3.3333333333333335 3 1 1000. Note that / always gives a float even when the division is exact: 10 / 5 is 2.0, not 2.

**Q234. What is the output of print("5" + "5") and print(int("5") + int("5")) ?**

The first prints 55, because + joins two strings. The second prints 10. This is exactly why int(input()) is needed - without it, adding two "numbers" from input just joins them.

**Q235. What is the output of print([1,2,3] * 2) and print([1,2,3] + [4]) ?**

[1, 2, 3, 1, 2, 3] and [1, 2, 3, 4]. Multiplying a list repeats it; adding two lists joins them. The same works for strings: "ab" * 2 gives "abab".

**Q236. What is the output of print(bool("False")) and print(bool(0)) ?**

True and False. Any non-empty string is truthy, even the text "False". This is why bool(input()) is a wrong way to read yes/no answers - compare the text instead.

**Q237. What is the output of a = [1,2,3]; b = a; b.append(4); print(a) ?**

[1, 2, 3, 4]. b = a does not copy the list, it only creates a second name for the same object, so the change is visible through both names. Use a.copy() or a[:] to make an actual copy.

**Q238. What is the output of print(0.1 + 0.2 == 0.3) ?**

False, because floats are stored in binary and 0.1 + 0.2 is actually 0.30000000000000004. Compare floats with a tolerance, for example abs(x - y) < 1e-9, or use the decimal module for money calculations.

**Q239. What is the output of print(type([]) == list, type(()) == tuple, type({}) == dict) ?**

True True True. Note that {} creates an empty dictionary, not an empty set - an empty set must be written as set().

**Q240. What does print(*my_list) do?**

The * unpacks the list, so print(*[1,2,3]) is the same as print(1, 2, 3) and outputs 1 2 3 separated by spaces instead of showing the list brackets.

**Q241. What is the difference between input() in Python 2 and Python 3?**

In Python 3, input() always returns a string, so conversion is needed. In Python 2, raw_input() returned a string while input() evaluated whatever was typed - which was unsafe. Other Python 3 differences: print is a function, integer division uses //, and strings are Unicode by default.

**Q242. Why is Python considered slower than C or Java?**

It is interpreted rather than compiled to machine code, it is dynamically typed so types are checked at runtime, and the GIL prevents true multi-threaded CPU parallelism. In practice this rarely matters, because heavy work is done by C-based libraries like NumPy, and development speed is much faster.

**Q243. What are Python's key advantages for a fresher to mention in an interview?**

Simple readable syntax that shortens development time, a huge standard library plus an ecosystem for web (Django, Flask), data (Pandas, NumPy) and automation, cross-platform support, a strong community, and being the default language for AI and machine learning work.

**Q244. What is the difference between a shallow "empty" check written as len(x) == 0 and if not x ?**

Both work, but if not x: is the Pythonic form and it also covers None. len(x) == 0 crashes with TypeError if x happens to be None, so prefer if not x: unless you specifically need to distinguish an empty container from None.